

Requirements Engineering: Use Case Diagrams

Prof. Luigi Libero Lucio Starace

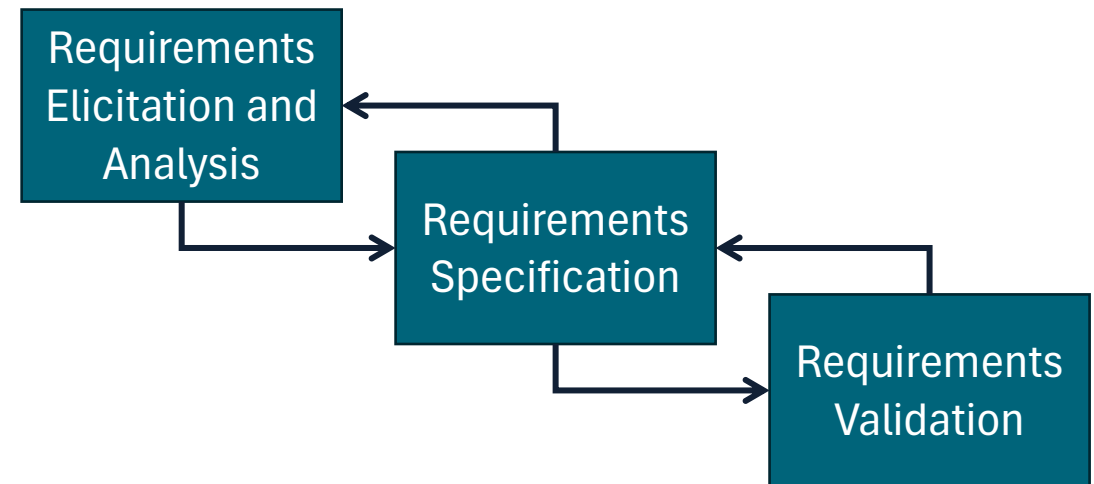
luigiliberolucio.starace@unina.it

<https://luistar.github.io>

<https://www.docenti.unina.it/luigiliberolucio.starace>

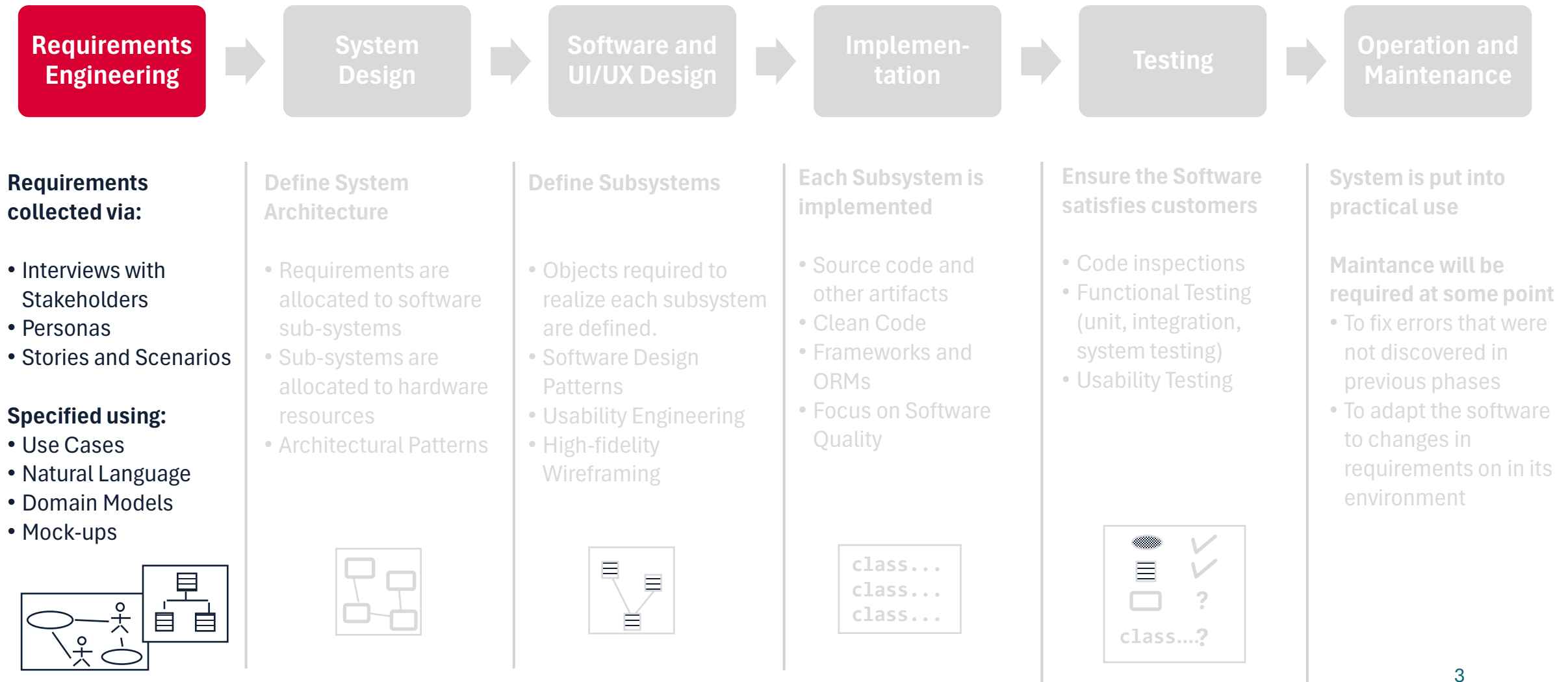
Previously, on Software Engineering

- We've seen what Requirement Engineering is and why it's important in the SDLC
- We got an overview of the main challenges and understand the main RE processes
- We looked into Requirements Elicitation and Analysis



The Requirement Engineering Process

The Software Life Cycle



Stories

- It is easier to relate to real-life examples rather than abstractions
- Stakeholders may not be very good at telling us system requirements, but they are generally more effective at describing how they actually do things or at envisioning how they might do things in a new way of working
- Stories are descriptions of how the system can be used for a particular task
 - What users do, what information they use, what output they need...

Use Case Diagrams



Use Cases

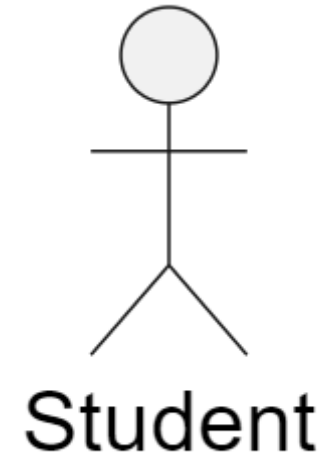
- Use Cases are a way to describe interactions between users and a system using a graphical model and structured natural language text.
- Are a key part of the Unified Modelling Language (UML), and can be used to represent the set of **functional requirements** of a system
- Use cases identify:
 - **Actors** – Categories of users (not necessarily humans) of the system
 - **Use Cases** – Types of interactions (or features) offered by the system
- Additional information on the interactions can be provided as (structured) textual descriptions, or by means of one or more semi-formal models (e.g.: UML Sequence Diagrams or Statecharts)

Use Cases

- Is a way to support communication with the client to define system functionalities
 - Should be as simple as possible
- Does not define «how» the system is implemented, but what the system should do
 - From the point of view of users (the system is a black box)
- Use cases are often documented using a high-level **Use Case Diagram (UCD)**

Actors

- Actors are represented using **stick figures**
- They represent external entities that interact with the **System Under Development (SUD)**
 - Classes of (human) users
 - Other systems
 - Physical Environment
- Every actor has a unique name
- Actors are more coarse-grained than Personas
 - One actor may be associated to multiple Personas



Use Cases

- Use cases are represented as named ellipses
- Correspond to **features** offered by the system, providing some **benefit** or **utility** to actors.
- Use cases model **functional requirements**.
- A use case abstracts many possible **scenarios** (sequences of actions) for a given feature
 - A scenario can be seen as an instance of a use case
 - A use case represents a class of scenarios that aim at using the same functionality
 - E.g.: there might be multiple ways to enroll in a course

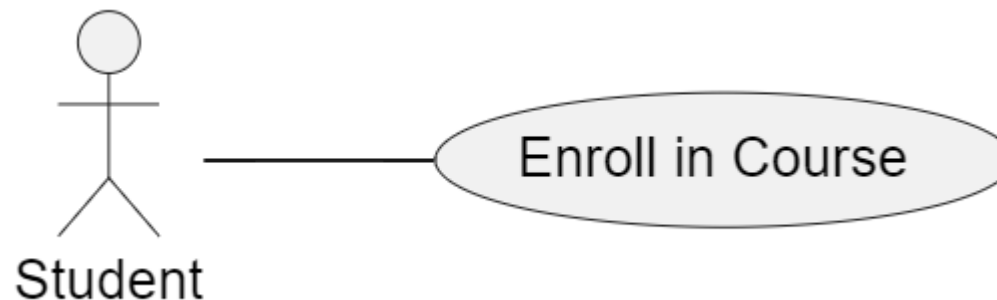


Heuristics to Identify Actors

- To identify actors, you may ask yourselves:
 - What groups of users are supported by the SUD in their job?
 - What groups of users perform the main features offered by the SUD?
 - What groups of users perform the secondary functions of the SUD, such as administration?
 - Will the SUD interact with external systems or software?
 - Each external system or software with which the SUD interacts will be an actor
- Actors do not correspond to a single entity, but rather represent a class of users that can have the same role
 - A user can have different roles in the same system
 - The Administrator of a website can also visit the website as a non-authenticated user

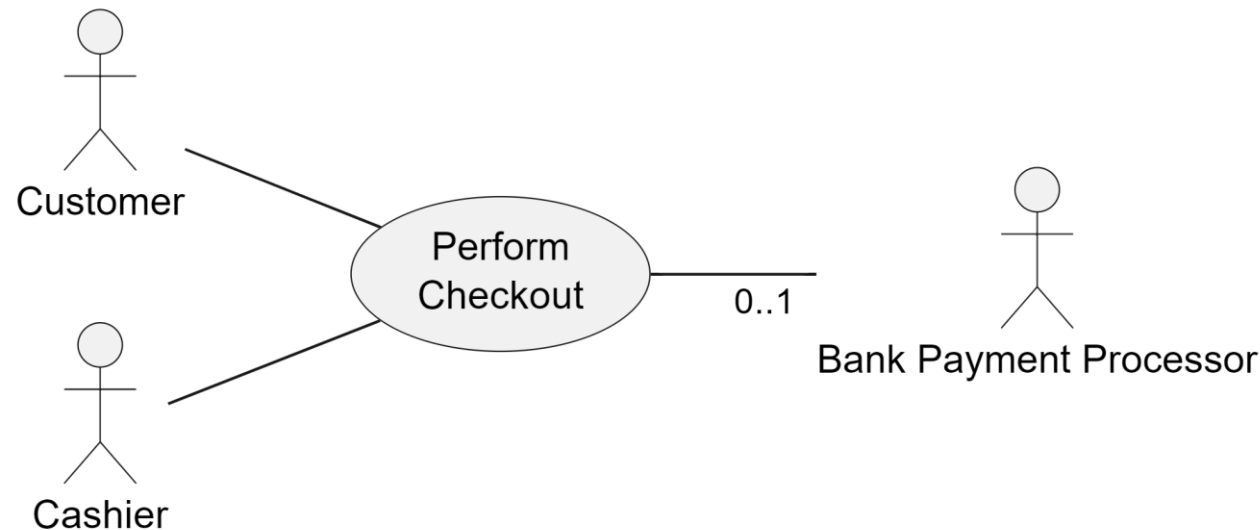
UCD: Associations

- In addition to actors and use cases, use case diagrams include different types of relations between them
- An association between an actor and a use case indicates that the actor can perform the use case
 - Graphically, it's represented as a line connecting an actor and a use case

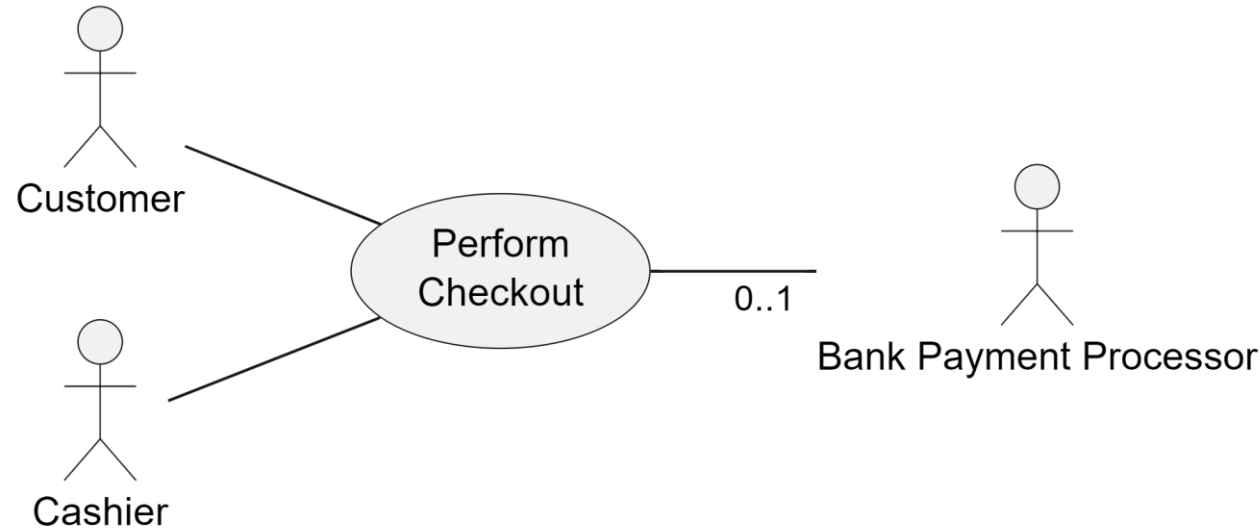


UCD: Secondary Actors

- A Use Case may be associated with multiple actors
- The semantics is that multiple actors need **collaborate** in some way to perform that use case
- Cardinalities are supported in UML 2.5



UCD: Secondary Actors



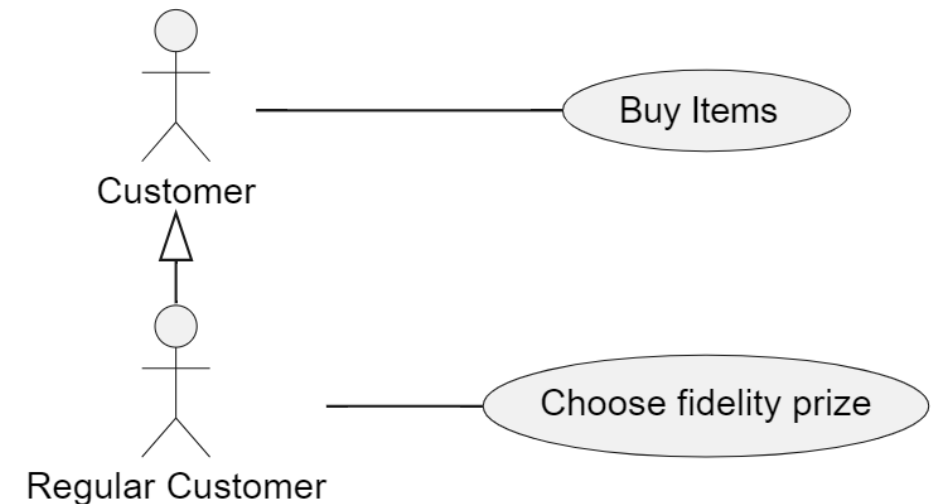
- Customer and Cashier need to cooperate to perform the checkout
- Optionally (cardinality is 0...1), a Bank Payment Processor (an external system) might be involved
 - E.g.: when the customer is paying using a credit/debit card

UCD: Secondary Actors

- UML UCD do not include facilities to specify **how** different actors involved in a use case
- Interactions modes and different responsibilities can be specified with additional descriptions (we'll see in a while)

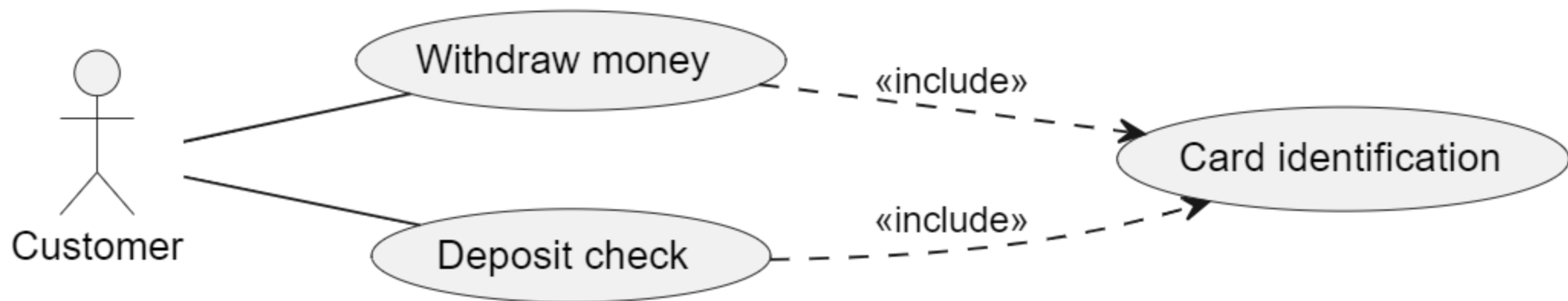
UCD: Actor Generalizations

- Generalization between actors can be applied when one actor is a sub-type of another actor
- Same concept and notation as in UML Class Diagrams
 - Graphically represented as an arrow with an «empty» head
- Specialized actor can perform any use case the parent can perform



UCD: Use Cases «include» Relation

- The «*include*» relationship is intended to be used when there are common parts of the behavior of two or more Use Cases.
- This common part is then extracted to a separate Use Case, to be included by all the base Use Cases having this part in common.

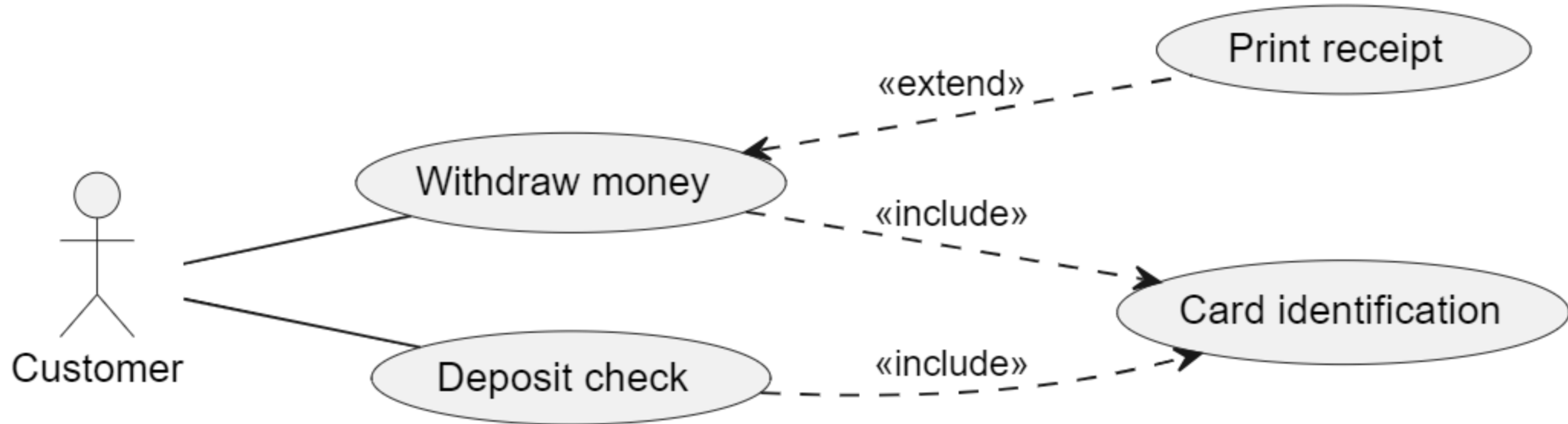


UCD: Use Cases «include» Relation

- As the primary use of the «include» relationship is for reuse of common parts, what is left in a base Use Case is usually not complete in itself but dependent on the included parts to be meaningful.
- This is reflected in the direction of the relationship, indicating that the base Use Case depends on the addition but not vice versa.
- This relation can be useful to:
 - **Decompose** a complex interaction into smaller, more manageable interactions
 - **Factor common sequences** of steps across different use cases

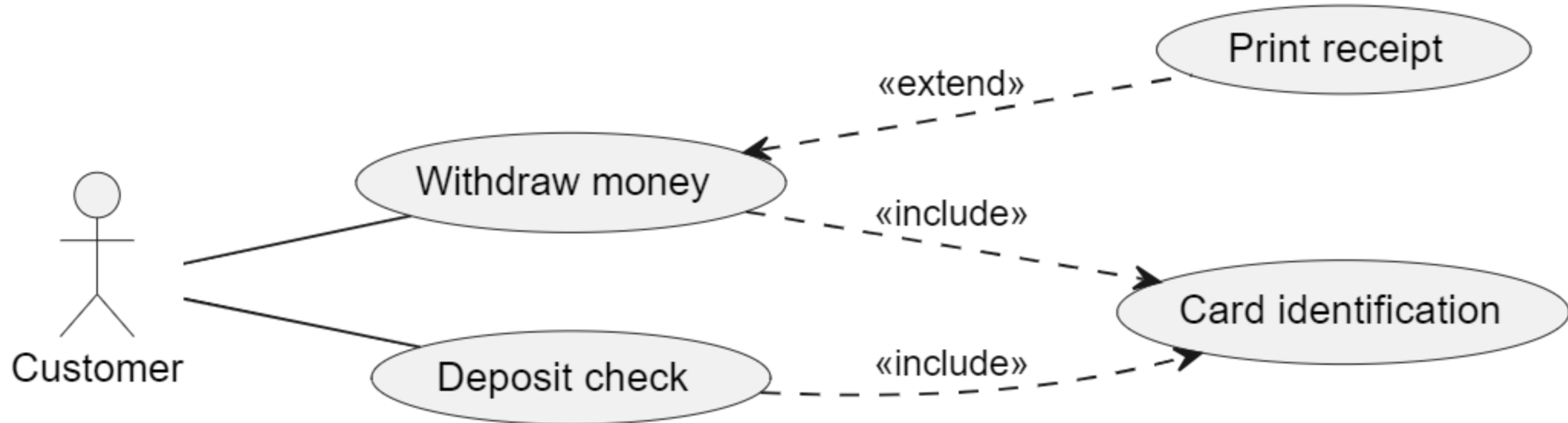
UCD: Use Cases «extend» Relation

- The «*extend*» relationship is intended to be used when there is some additional behavior that **may** be added, **possibly conditionally**, to the behavior defined in one or more Use Cases.



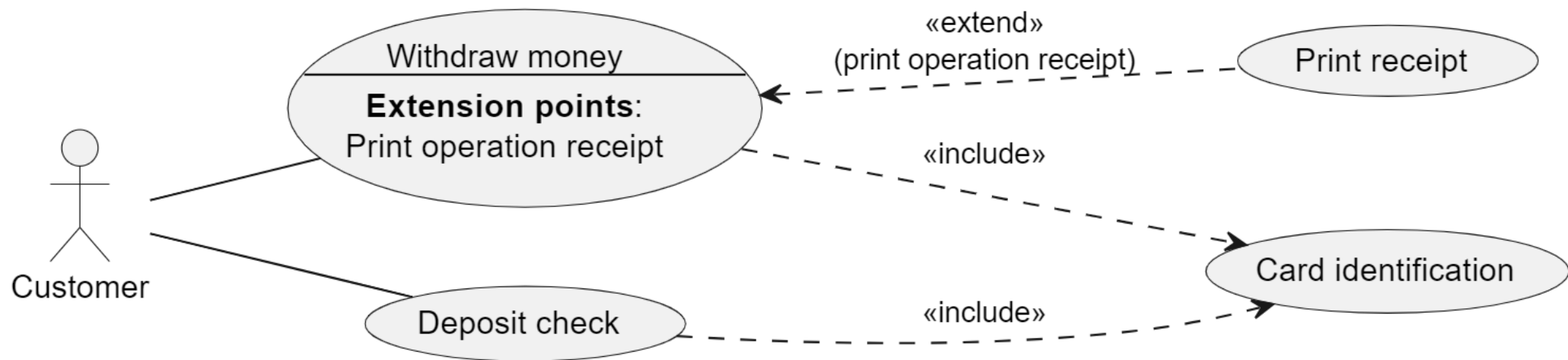
UCD: Use Cases «extend» Relation

- The extended Use Case is defined independently of the extending Use Case and is meaningful independently of the extending Use Case
- On the other hand, the extending Use Case typically defines behavior that may not necessarily be meaningful by itself.



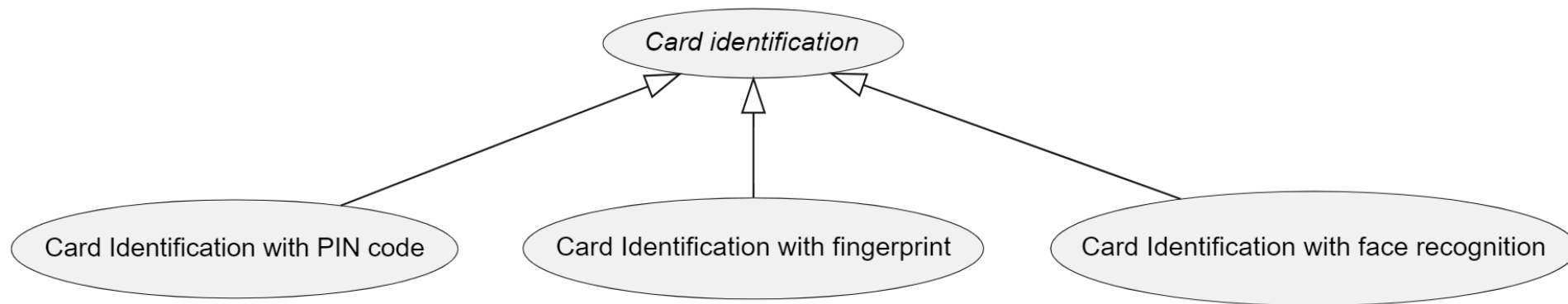
UCD: Use Cases Extension Points

- Extension points identify the point in the base Use Case where the behavior of an extension Use Case can be inserted
- They can be useful when a Use Case can be extended in multiple points

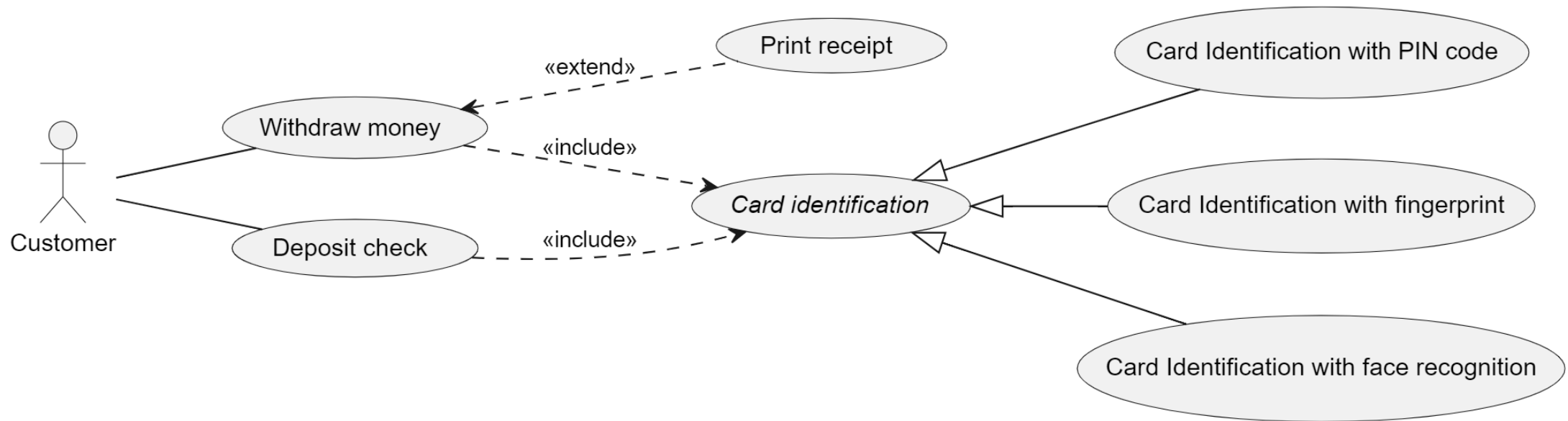


UCD: Use Cases Generalization

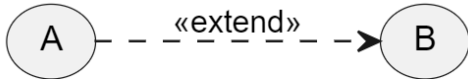
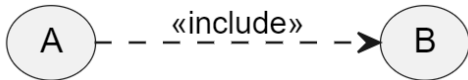
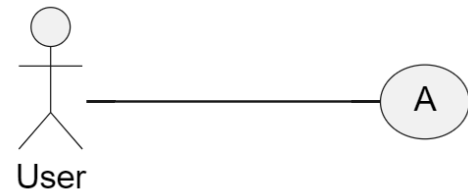
- Generalization between use cases is intended to be used when a Use Case is a specialization of another
- Differently from «extend» dependencies, there is no precise points in which the specialized use cases deviate from the parent
- Specializations can be very different w.r.t. the parent use cases.
- Notation is the standard UML notation for specialization



UCD: Use Cases Generalization

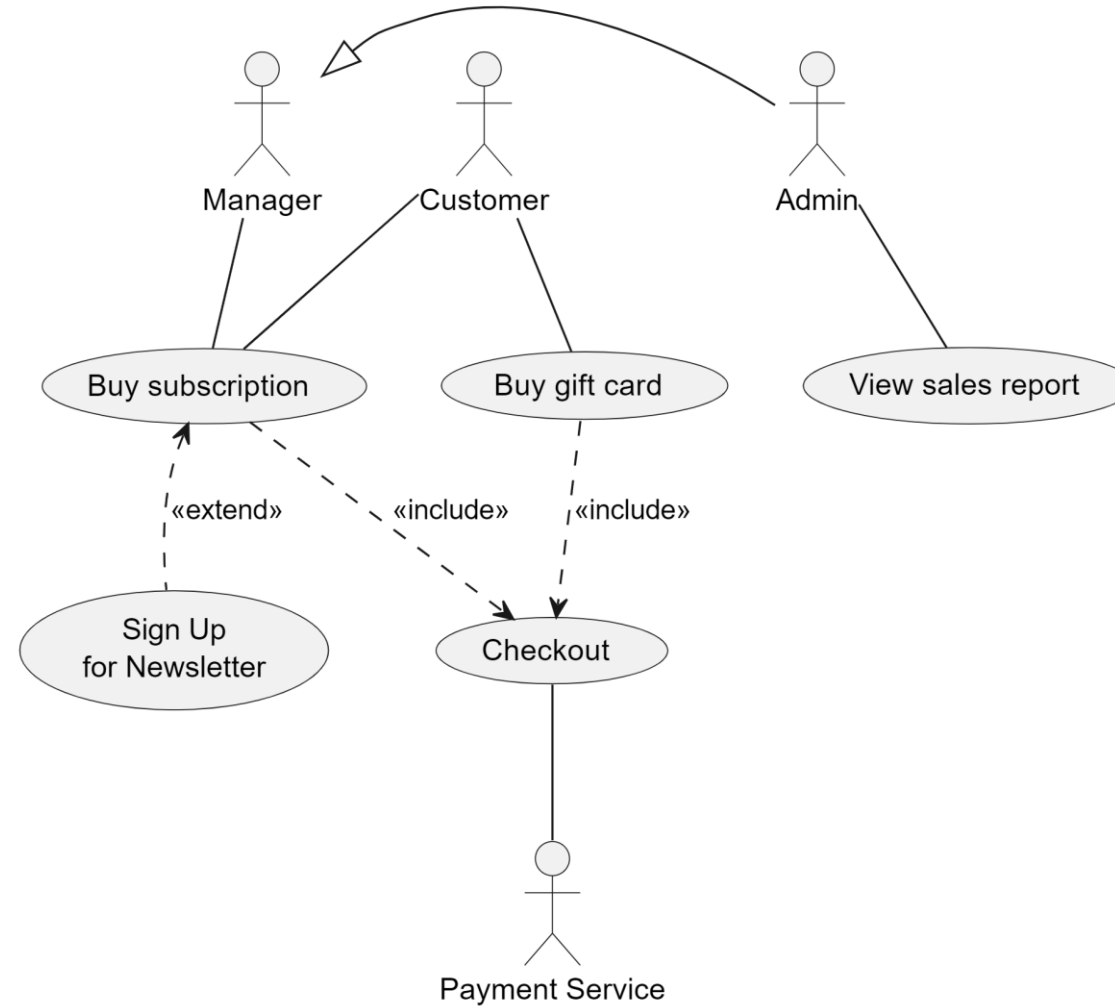


UCD: Relations Overview

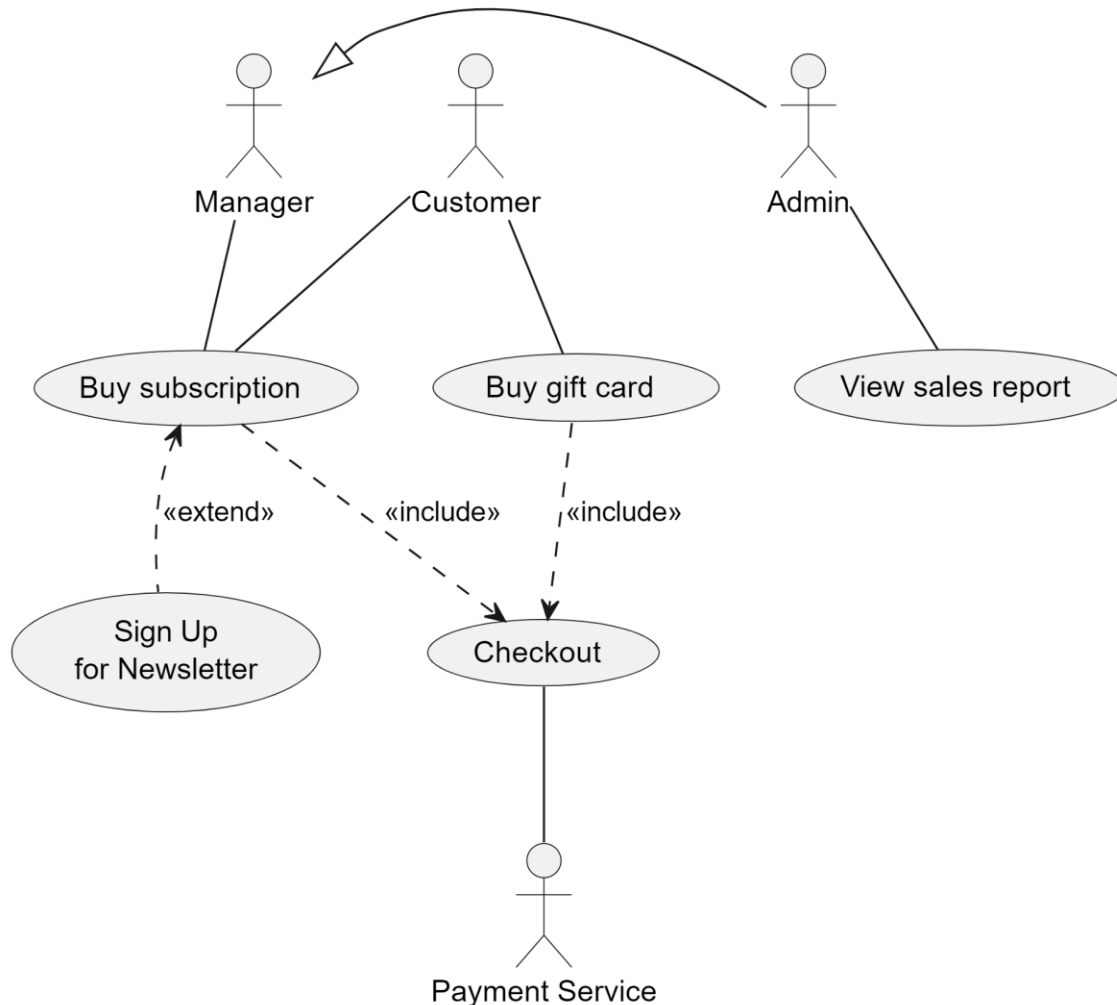


- **Association:** an actor is associated to a use cases
 - User can perform UC A
- **Include:** Common behaviour is factorized
 - UC A includes steps defined in UC B
- **Extend:** Identifies (possibly optional) variations.
 - UC A can include the sequence of steps in UC B
- **Generalization:** can be applied to actors and use cases.
 - B inherits the behaviour of A
 - A can be replaced with an instance of B

Reading a Use Case Diagram



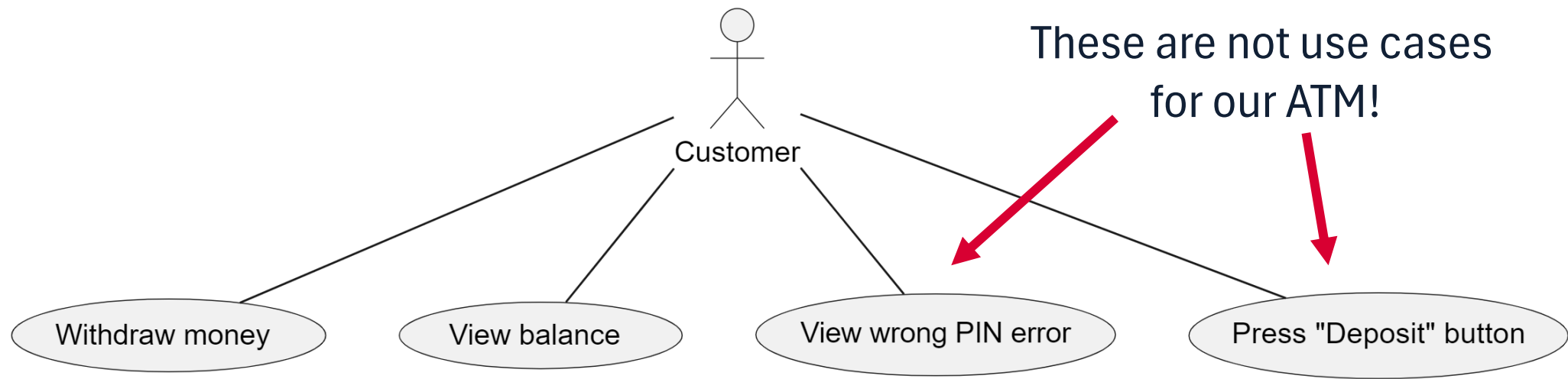
Reading a Use Case Diagram



- Customers can use the system to buy a gift card or a subscription
 - Both use cases include a common checkout behaviour (sequence of steps) for payment
 - This sequence of steps requires collaboration with an external Payment Service
 - To Buy a subscription, the collaboration of a manager (or an admin) is required (e.g.: they may need to approve the purchase)
 - Optionally, when buying a subscription, the customer can sign up for the newsletter
- Admins are the only one that can view sales reports

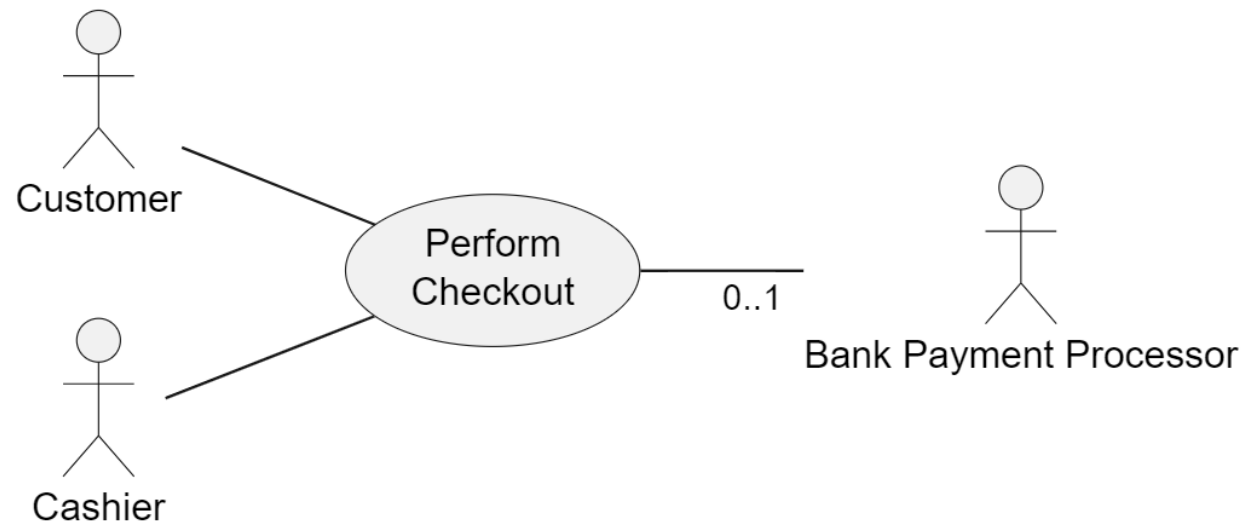
Use Cases: Rookie Mistakes – I

- Use cases should provide some benefit to the actor, help the actor complete his job or achieve some goal.
 - Typically, Use Case names should include a verb
 - Typically, Actor names should be nouns



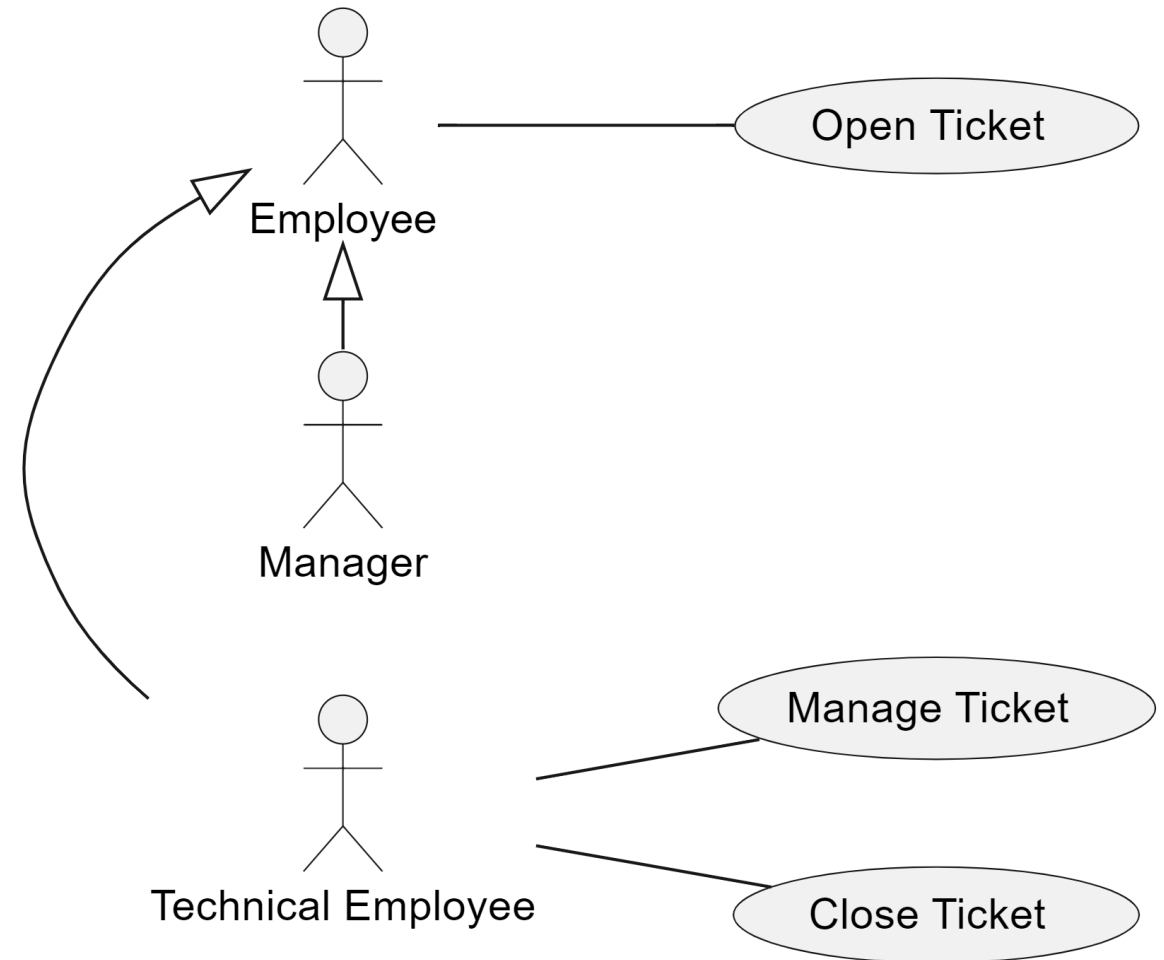
Use Cases: Rookie Mistakes – II

- If two actors are associated with the same use case (with a non-zero cardinality), it means that the two actors are involved (and need to collaborate) in each instance (scenario) of that use case
 - It does not mean that both actors can perform that use case independently!



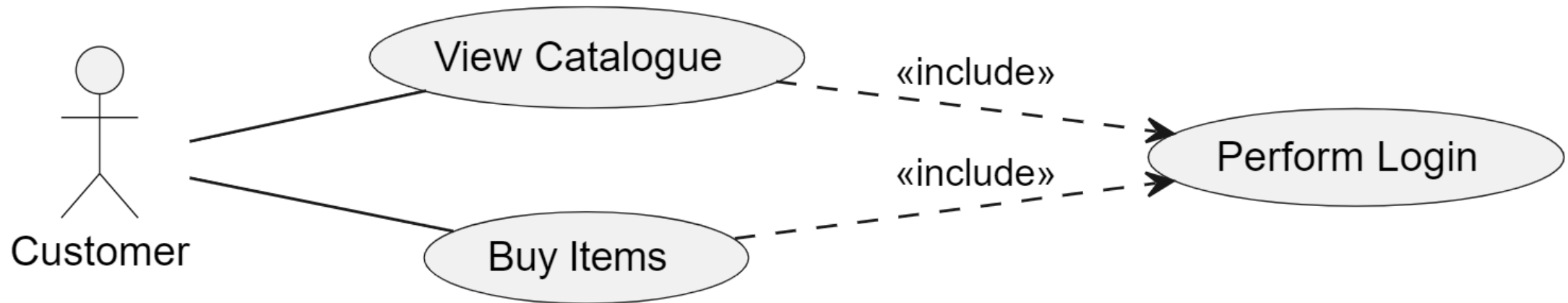
Use Cases: Rookie Mistakes – III

- Beware of improper generalization
 - Every actor should have their own use case(s)
 - Specialized actors can already perform all the use cases of their ancestors
 - If the specialized actors does not have some use cases of their own, the generalization might be useless, or some use cases might be missing



Use Cases: Rookie Mistakes – IV

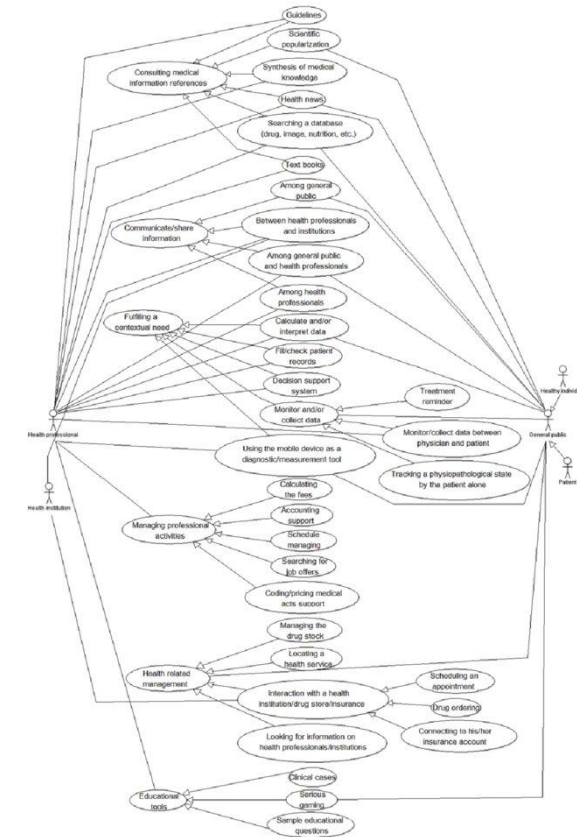
- «*include*» is not a good way to represent temporal relations



- The above diagram indicates that **Customer** needs to perform the sequence of steps in the **Perform Login** UC every time they want to **Buy Items** or **View Catalogue**!
- **Preconditions** in the text description of the UC should be used instead

Use Cases: Rookie Mistakes – IV

- Use case diagrams should not get too complex and messy
 - Use relationships between use cases and generalizations between actors with moderation
 - The modeling should be at a sufficiently high degree of abstraction
 - You need to be orderly (try to avoid intersecting lines, etc...)
- A **complex diagram** is an indication of a **bad analyst**.



Yasini, M., & Marchand, G. (2015).
Toward a use case based classification
of mobile health applications.

Specifying Use Cases

- The UCD provides a very high-level overview of the functional requirements of the systems. It is not detailed enough to establish system requirements
- For **each** UC in the UCD a detailed specification is needed
- The goal is to specify every aspect and detail of the interaction, from the Actor's point of view.
 - Each possible scenario and variation should be described

Text Descriptions of a Use Case : Scenarios

- A use case description generally include:
 1. A description of what the system and users expect when the use case begins
 2. A description of the normal flow of events in the Use Case (**main scenario**)
 3. A description of what can cause errors and how the resulting problems can be handled
 4. A description of the state of the system after the Use Case is complete.

Use Case Formats

Use cases can be written in different formats and levels of formality:

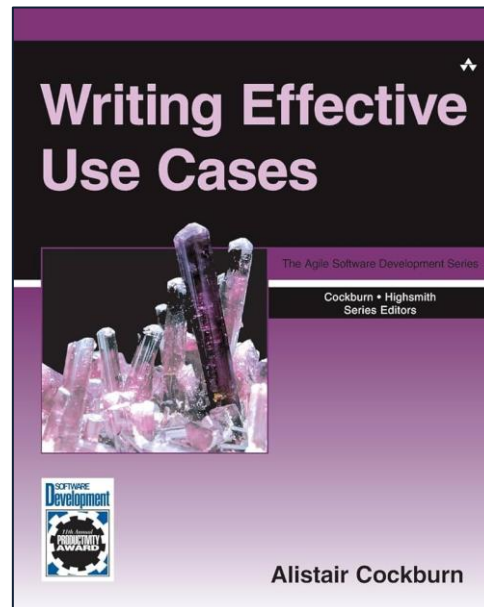
- **Brief:** Terse one-paragraph summary, usually of the main success scenario.
- **Casual:** Informal paragraph format. Multiple paragraphs that cover various scenarios.
- **Fully-dressed description:** All steps and variations are written in detail, and there are supporting sections, such as preconditions and success guarantees.

Use Case Formats

- **Brief** and **Casual** descriptions can be used in the early stages of requirements specification, to get a quick sense of subject and scope
- **Fully-dressed** descriptions may be developed later on, to serve as a basis for a contract and specify in greater detail the behaviour of the system to be developed

Fully-dressed Use Case Descriptions

- Different formats for fully-dressed use case descriptions have been proposed
- We'll see a template based on the one proposed by **Alistair Cockburn**



Cockburn's Template

USE CASE #X	Name of the Use Case			
Goal in Context	Description of the objective of this UC			
Preconditions	All the conditions that must apply to start the UC			
Success End Condition	State of the system if the UC was successful			
Failed End Condition	State of the system if the UC failed			
Primary Actor	Primary actor of the UC			
Trigger	Action of the primary actor that initiates the UC			
Main Scenario	Step n.	Actor 1	Actor n	System
	1	Trigger action		
	2			Response
	..	Action 2		
	..		...	...
	n			Final action

Cockburn's Template (cont.)

Extension #1 (short description)	Step	Actor 1	Actor n	System
	x <condition>	...	...	...
	...	..	...	...
	...			Final action (possibly return to a step of the main scenario)
Extension #n (short description)	Step	Actor 1	Actor n	System
	y <condition>	...	...	...
	...	..	...	...
	...			Final action
Open Issues	List all the aspects that still need to be clarified. At the delivery of the doc must be empty			

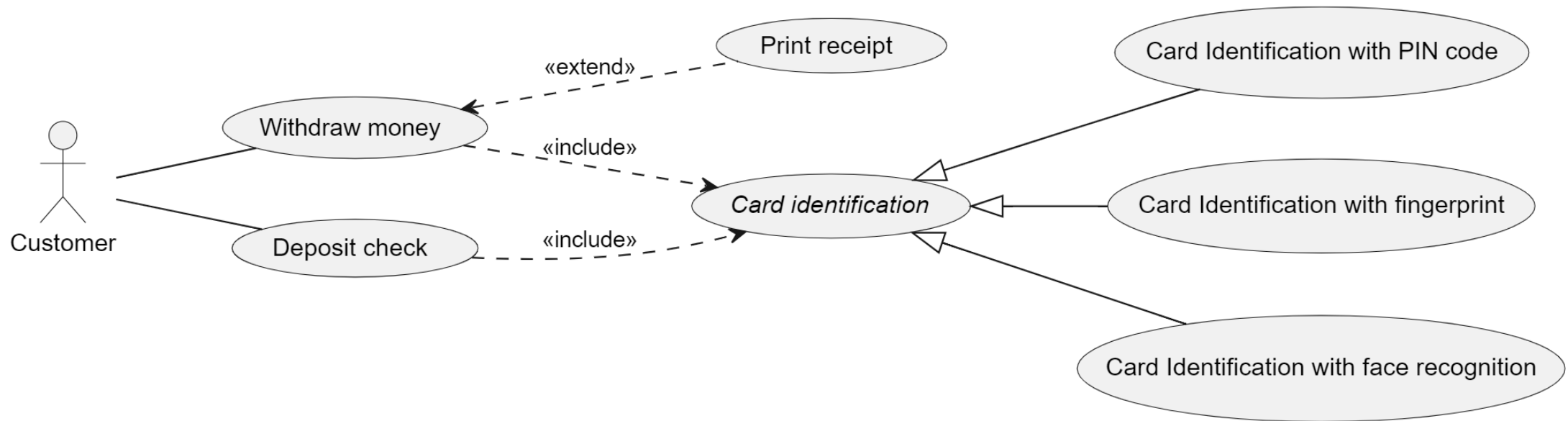
Main Scenario And Extensions

- The main scenario is the sequence of actions that occurs when all in the use case goes smooth as intended
- However, there may be different ways to perform an use case
 - Users can authenticate themselves by using the PIN or a fingerprint scanner
 - An error might occur at some point
- When defining the functional behaviour of the system, it is important to describe also these alternative sequences of actions that can happen when performing a use case
 - This is done using **Extensions**
 - Typically, there's way more text in the Extensions rather than in the Main Scenario

Example



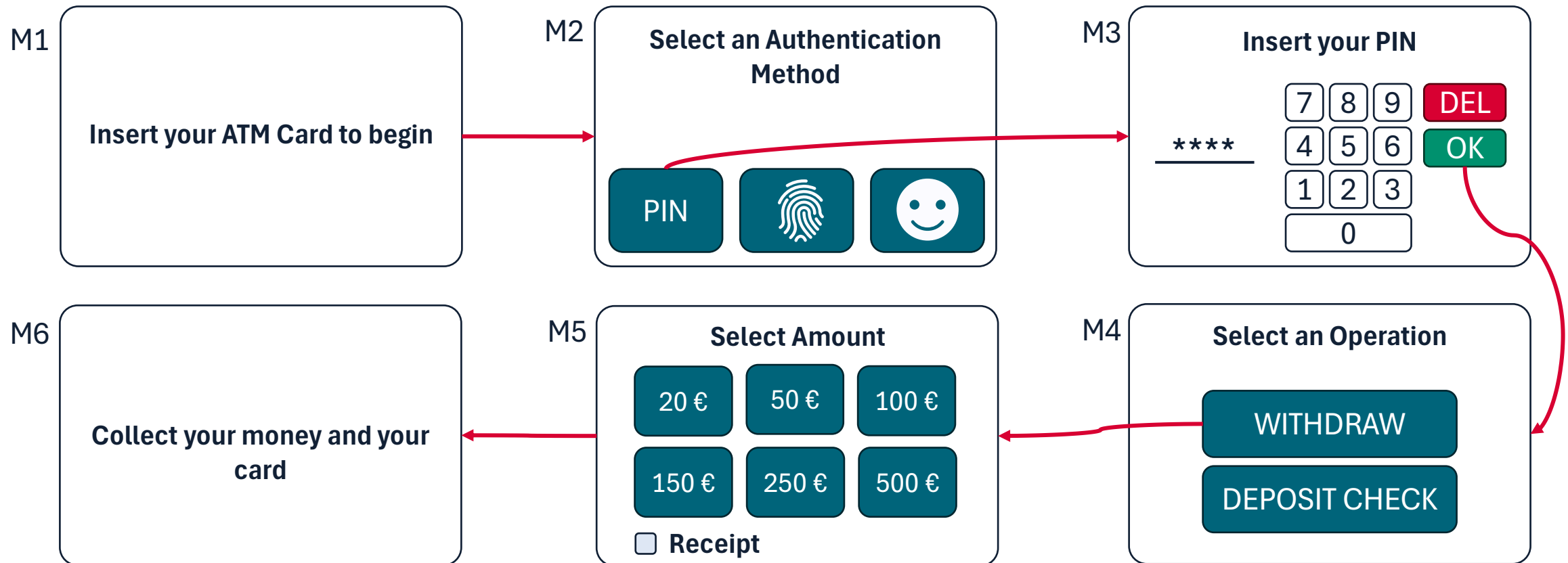
Example: ATM System



- Suppose we want to describe the **Withdraw money** use case, using a fully-dressed format

Example: Mockups

- It may be useful to design some mockups of the system



Example: Fully-dressed Use Case

USE CASE #1	Withdraw money		
Goal in Context	A customer wants to withdraw money from the ATM		
Preconditions	The customer has an account at the Bank and owns a bank card. He has 500\$		
Success End Condition	The system keeps track of the withdrawal operation and erogates the requested money		
Failed End Condition	No transaction is made		
Primary Actor	Customer		
Trigger	Customer walks up to the system and touches the screen to activate it		
Main Scenario	Step n.	Customer	System
	1	Touches screen	
	2		Shows M1
	3	Inserts card	
	4		Shows M2

Example: Fully-dressed Use Case

USE CASE #1	Withdraw money		
Main Scenario	Step n.	Customer	System
	5	Touches «PIN» button	
	6		Shows M3
	7	Inserts PIN	
	8		Shows M4
	9	Touches «Withdraw» button	
	10		Shows M5
	11	Touches «50 €» button	
	12		Erogates money, Ejects card, Shows M6

Example: Extensions

- What can go wrong?
 - PIN might not be correct
 - Customer might not have enough money in their account
 - ATM might not have enough cash reserves to erogate the required money
 - Card might be flagged as stolen
 - Card might be unreadable
 - ...
- What could go differently?
 - Customers might authenticate themselves using their fingerprint or face recognition
 - Customers might opt-in to get the printed receipt

Example: Extensions

- Each of these scenarios should be detailed using extensions

Extension #1 (customer inserts an invalid PIN)	Step	Customer	System
	7a <wrong PIN is inserted>	Inserts PIN	
	8a		Shows M7 and terminates UC

M7

**Invalid PIN – Authentication denied.
Recover your card from the tray.**

Example: Extensions

Extension #2 (customer does not have enough money)	Step	Customer	System
	11b	Selects «500€» button	
	12b		Shows M8
	13b	Clicks ok	
	14b		Return to step 10 of the Main Scenario

M8

Your balance is smaller than the amount you are trying to withdraw. Select a smaller amount.

OK

Guidelines for writing effective use cases

- In addition to the original Alistair Cockburn book, there are many other resources including guidelines for writing effective use cases:
 - Wei, B., Deng, L., Wang, Y. (2022). A Practical Style Guide and Templates Repository for Writing Effective Use Cases. In: Lee, R. (eds) Software Engineering Research, Management and Applications. SERA 2022. Studies in Computational Intelligence, vol 1053. Springer, Cham.
https://doi.org/10.1007/978-3-031-09145-2_5
- This paper also proposes a set of templates of generic use cases
 - <https://github.com/Washingtonwei/use-case-style-guide>
- Finally, there is also a complete example (SuperFrog Scheduler)
 - https://docs.google.com/document/d/1PBDgqCbMPpyrAWZnob_OxDecOPoyyOtV_sJeeObh62E/edit?tab=t.0